

WAL and Binlog

Thursday, August 20, 2026 10:06 AM

This is a classic **Senior vs. Junior** question.

If you answer with just **"WAL is for crash recovery, binlog is for replication,"** you pass. But if you want the **Principal Engineer** answer, you need to explain the **storage engine level (Physical vs. Logical)**, the **timing of writes**, and the **replication topology** implications.

Here is your complete deep-dive breakdown, structured like a battle-tested architect's guide.

The 10-Second TL;DR (The Interview Hook)

- **WAL (Write-Ahead Log)**: The database's **insurance policy** against crashes. It lives at the **storage engine** level. It logs **physical changes** to disk pages.

- **Binlog (Binary Log)**: The database's **tape recorder** for sharing data. It lives at the **server** level. It logs **logical changes** (SQL statements or row mutations) for replication and point-in-time recovery.

Part 1: The "NCRT" Level Differences (The Basics)

| Feature | Write-Ahead Log (WAL) | Binary Log (Binlog) |

| :--- | :--- | :--- |

| **Primary Purpose** | **Crash Recovery** (Durability). Ensures committed data isn't lost if the power fails. | **Replication & PITR** (Point-In-Time Recovery). Sends data to replicas or restores to a specific timestamp. |

| **Layer** | **Storage Engine** (e.g., InnoDB). | **Server Layer** (e.g., MySQL Server). |

| **What it logs** | **Physical/Page-level** changes (e.g., "Changed byte 0x45 at offset 1024 on page 7"). | **Logical** changes (e.g., "INSERT INTO users VALUES ('Bob') OR 'Row image: {id:1, name:'Bob'}"). |

| **When it writes** | **BEFORE** the data page is modified in memory (Write-Ahead principle). | **AFTER** the transaction commits (usually), or alongside the commit. |

| **Idempotency** | **Not idempotent**. Replaying a physical page change twice corrupts data. | **Idempotent**. Replaying `REPLACE INTO` or row-based changes multiple times yields the same final state. |

Part 2: The Deep-Dive Mechanics (How they actually work)

Interviewers care about the **order of operations**. If they ask **"What happens during an `UPDATE` in MySQL with InnoDB?"**, here is the exact choreography:

- Buffer Pool**: Database reads the disk page into memory and modifies the row.
- WAL Write (InnoDB Redo Log)**: Before the page is flushed to disk, the physical change is **forced to the WAL** on disk (`fsync`). **This is the "Write-Ahead" guarantee.**
- Binlog Write**: The logical change is written to the Binlog on disk.
- Prepare/Commit (XA Protocol)**: MySQL uses a two-phase commit (2PC) between the WAL (Storage engine) and Binlog (Server) to ensure they match.
- Data Flush**: Eventually, the memory pages are asynchronously flushed to the actual data files (`.ibd` files).

The Senior Insight: The WAL is written **during** the transaction. The Binlog is written **at commit time**. If the database crashes between step 2 and step 3, the WAL lets InnoDB roll forward committed changes, but the Binlog doesn't have the change—so replicas stay safe (they never get the change).

Part 3: Physical vs. Logical (The "Hidden Gold" of this question)

This is the **killer distinction** that puts you above 90% of candidates.

- **WAL (Physical):** Because it logs physical page offsets, it is **extremely compact and fast**. You don't have to parse SQL or figure out which rows changed. You just literally overwrite the bytes on the disk. This is why WALs are used for **crash recovery**—it's just raw, fast byte-shifting.

- **Binlog (Logical):** Because it logs rows/SQL, it is **independent of the storage engine**. You can use a Binlog to replicate from MySQL (InnoDB) to a completely different system like Kafka, Elasticsearch, or a Data Warehouse.

- **Row-based binlog:** Logs the "before" and "after" image of the row.

- **Statement-based binlog:** Logs the exact SQL.

The Catch: Because Binlog is logical, it is **larger and slower** than the WAL. Writing a massive `UPDATE` that modifies 1 million rows results in a tiny WAL entry (just a few page changes) but a **massive** Binlog entry (1 million row images).

Part 4: The Advanced Interview Twist (The "MySQL 2PC" Trap)

Interviewer: "What happens if the server crashes right after writing the WAL but BEFORE writing the Binlog?"

The Junior Answer: "The data is lost."

The Senior Answer:

> **Actually**, in MySQL/InnoDB, the WAL and Binlog are synchronized using an internal distributed transaction (XA). If the WAL is written but the Binlog isn't, the transaction remains in a 'PREPARE' state upon crash recovery. During the recovery phase, InnoDB will look at the Binlog. Since the Binlog doesn't have the commit record, InnoDB **rolls back** the transaction, even though the WAL has the changes. This ensures the Master and Replicas never diverge."

Part 5: The Architectural Implications (Why you care)

| Aspect | WAL (InnoDB Redo Log) | Binlog (MySQL Server) |

| :--- | :--- | :--- |

| **Disk Space** | **Fixed size** (Circular buffer). It overwrites old logs automatically once the dirty pages are flushed to disk. | **Append-only** (Grows indefinitely until you `PURGE` or expire it). |

| **Group Commit** | Can group multiple physical page changes into one disk write. | Can group multiple logical transactions into one disk `fsync` to improve throughput. |

| **Replication** | Cannot be used for replication between different versions or storage engines. | **Universal.** Used for Master-Slave, Master-Master, and CDC (Change Data Capture) to Kafka. |

| **Recovery Speed** | **Extremely Fast.** Re-applies physical bytes at the page level. | **Extremely Slow.** Re-applying a Binlog for crash recovery would require re-parsing SQL/rows and executing them through the query engine. |

Part 6: The "PostgreSQL" Curveball

If the interviewer mentions PostgreSQL, pivot hard.

PostgreSQL **does not have a separate Binlog**. In PostgreSQL, the WAL **is** the binlog.

In Postgres, the WAL is used for **both** crash recovery **AND** replication (via streaming replication).

This proves your deep understanding:

- **MySQL's philosophy:** Separate the Storage Engine (InnoDB WAL) from the Server (Binlog) for flexibility.

- **PostgreSQL's philosophy:** Tightly couple the log to the storage engine, so you don't need the 2PC overhead. You just ship the physical WAL to replicas. (Though logical decoding was added later to mimic a binlog for CDC).

The "Elevator Pitch" for your Interview

When they ask **"Difference between WAL and Binlog?"**, structure your answer like this:

> **"They operate at entirely different layers and times. The WAL is a **physical, fixed-size redo log** at the storage-engine level, written **before** data pages are modified, purely to guarantee durability and fast crash recovery.**

>

> **The Binlog is a **logical, append-only sequence of row changes** at the MySQL server level, written **at commit time**, designed specifically for cross-cluster replication and point-in-time recovery.**

>

> **The critical architectural consequence is the **2PC dance** between them—if the Binlog isn't written, the recovery process will roll back the WAL changes to keep replication consistent. This also means Binlog grows forever until manually purged, while WAL is a circular buffer that overwrites itself.**

>

> **If I were using PostgreSQL, however, I'd note that it combines both jobs into a single WAL to avoid that 2PC overhead entirely."**

Why this works: You defined both, contrasted the technical layers, explained the order of operations, called out the replication vs. crash recovery distinction, and dropped a cross-database comparison to show you aren't just a framework user—you understand database internals. That is a guaranteed "Strong Hire" checkmark.